Overview

Zihad Patwary

Paper title

When a Restart Corrupts the Result: Duplicate and Missing Records in Dataset Enrichment Pipelines

Thesis

A restart can duplicate or omit records when output and completion are saved separately. I compare replay-safe writes with atomic completion, measuring correctness and recovery cost. The paper is in the six numbered tabs; the remaining tabs provide supporting reference material.

Sections

Section	Core claim
1. The problem	Separate output and completion writes can disagree after a crash
2. Who is affected	Downstream users may receive plausible but incomplete or duplicated data
3. Evidence	Documentation establishes the risk; a synthetic dry run reproduced both failures
4. Existing solutions	Idempotent merge versus atomic progress in the commit
5. Key unknown	Is Delta's transaction id as safe as merge, and cheaper?
6. Individual scope	Four strategies, defined crash points, one laptop
NSF 26-512 summary	What the published opportunity says, section by section
Sources and acronyms	Source addresses and first-use definitions

1. The problem

What problem are you investigating?

Main point

I am investigating why a dataset can come out wrong after a pipeline crashes and restarts. Saving the rows and recording that a batch is done are two separate steps. A crash between them leaves the restart with a wrong record of which work is finished, and it acts on that record without raising an error.

Define the terms

- **Enrichment:** adding derived fields to every record.
- **Worker:** a separate process that takes a batch (task) from a queue, enriches it, and saves it.
- **Progress marker:** a file saying "task X is done"; restart skips marked tasks.

When it happens

The failure considered here has these conditions:

1. Work is split into batches and run by one or more workers.
2. A worker process terminates abruptly before completing its write sequence.
3. The job is restarted by an operator or a scheduler retry.
4. Output and progress are separate writes; replaying an append adds another copy.

Worked example: an earthquake data feed

Case	What the worker does	Result after restart
Normal	Reads a saved United States Geological Survey (USGS) feed in comma-separated values (CSV) format; sorts by event identifier; forms 250-row batches; adds depth, magnitude and region categories	Every earthquake appears exactly once
Failure A: save, then mark	Appends task 4's 250 rows, then dies before writing its marker	No marker, so task 4 reruns: **250 duplicate rows**, no error
Failure B: mark, then save	Writes the marker, then dies before the rows are saved	Restart trusts the marker and skips task 4: **250 missing rows**, all markers say done

In the normal run, each worker saves its 250 rows and then writes its marker, so the saved rows and the markers always agree. The danger is the moment between those two writes. In Failure A the rows are saved but the marker is missing, so the restart believes task 4 never ran and saves it again. In Failure B the marker exists but the rows do not, so the restart believes task 4 finished and skips it. This is a hypothetical scenario, not a reported USGS incident.
Data format: https://earthquake.usgs.gov/earthquakes/feed/v1.0/csv.php

Where the crash lands decides the damage

Strategy	Step 1	Crash	Step 2 (never ran)	After restart
Save, then mark (naive_append)	Append 250 rows	X	Write marker	**250 duplicates**: no marker, task reruns
Mark, then save (mark_first)	Write marker	X	Append 250 rows	**250 missing**: marker says done
One atomic commit (delta_txn, not yet tested)	Rows and done record in one commit	Commit boundary	Check committed task	**Expected: skip a committed replay; retry an uncommitted task**

My synthetic dry run produced exactly these counts for the first two rows; the third row is not yet tested. The third row closes the gap instead of tolerating it: Delta transaction identifiers save the rows and the "done" record in one atomic commit, meaning both are written or neither is, so a crash cannot leave one without the other. A key-based MERGE (update the matching row, otherwise insert it) takes a different route: a batch can be saved twice without creating copies, but its marker is still a separate write.
https://github.com/delta-io/delta/blob/master/PROTOCOL.md#transaction-identifiers

Boundaries

- **In scope:** worker-process termination during a batch, then full-job restart, on one machine and one table.
- **Out of scope:** wrong enrichment logic, corrupted input files, network partitions between machines, malicious actors, multi-table transactions.

Why it is hard to spot

Duplicates can pass a schema check. Reference-output and record-key checks reveal missing, repeated or wrongly enriched records.

2. Who is affected

Who or what is affected by it?

Main point

Operators and downstream users can receive incorrect records despite a successful job and valid schema.

Affected parties and consequences

Affected party	What goes wrong	Tie back to the earthquake example
Data engineers who run the job	Repeated processing and diagnosis	Find the extra 250 rows from replaying task 4
Scientists and analysts using the output, such as a university seismology research group	Duplicates inflate counts; missing rows hide events	A regional earthquake count can be inflated or reduced if affected records belong to that region
Artificial intelligence (AI) and machine learning (ML) training datasets	Repeated examples can receive extra weight; missing records can reduce coverage	A duplicated event may be sampled twice; a skipped event is unavailable for training
The pipeline system itself	Progress no longer matches output	In Failure B every task has a marker while 250 records are absent

Why the consequences matter

Repeated enrichment wastes computation; diagnosing discrepancies takes time. Effects on a model depend on sampling and training choices. These are plausible consequences, not measured outcomes.

Limits of these claims

The National Science Foundation (NSF) opportunity motivates reliable scientific datasets. It establishes neither an organizational incident nor model harm.
https://www.nsf.gov/funding/opportunities/ai-datasets-unlocking-dataset-value-ai-enabled-scientific-discovery/nsf26-512/solicitation

3. Evidence

What evidence do you currently have that it is a real problem?

Main point

Documentation establishes a recognized retry problem, and my synthetic dry run reproduced both failure modes. Behavior on real data and performance still need measurement.

Documented facts (from sources)

Claim	Source	What it proves	What it does not prove
NSF AI Datasets includes reliable preparation pipelines and dataset integrity among its priorities.	NSF solicitation	Funders want reliable pipelines	That restart duplicates actually happen
The Delta protocol states that incremental systems "need to record what progress has been made, in order to avoid duplicating data in the face of failures and retries during a write."	Delta Lake protocol, Transaction Identifiers	The mechanism is a recognized design concern; a feature exists for it	How often it occurs
Airflow warns: "Do not use INSERT during a task re-run, an INSERT statement might lead to duplicate rows in your database. Replace it with UPSERT" (update or insert).	Apache Airflow best practices	Re-runs are normal, and duplicates are a known risk	That this implementation has failed
Spark explains that replayable sources and idempotent sinks	Spark Structured Streaming guide	Stream processors treat it as a core requirement	Behavior outside Spark

support end-to-end exactly-once processing.			

None of these sources measures this implementation or how often the failure occurs.
NSF: https://www.nsf.gov/funding/opportunities/ai-datasets-unlocking-dataset-value-ai-enabled-scientific-discovery/nsf26-512/solicitation
Delta: https://github.com/delta-io/delta/blob/master/PROTOCOL.md#transaction-identifiers
Airflow: https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html
Spark: https://spark.apache.org/docs/latest/streaming/getting-started.html

Observations and measurement status

- On October 18, 2026, all 9 automated tests passed (Python 3.12.1, deltalake 1.6.6). I then ran the synthetic dry run: 2,000 generated records in the USGS format (source version 8e818e91a678832d), split into 8 tasks, with a crash injected at task 4, using 1 and 2 workers.
- Observed: with no crash, all three strategies ended with 0 missing, 0 duplicate and 0 incorrect rows. After the crash and restart, naive_append had 250 duplicate rows and mark_first had 250 missing rows, with 1 and 2 workers alike, while idempotent_merge had none of either. Provenance was 100% complete in every run.
- These are single runs on synthetic data. Results on the live USGS feed, repeated trials and timing have not been measured yet.

Assumptions and missing evidence

- **Assumption:** enough practical pipelines separate output and progress writes for this comparison to be useful. No prevalence survey is supplied.
- **Assumption:** failures can occur within the vulnerable interval. Deliberate termination tests the consequence, not its natural probability.
- **Missing:** a verified production incident, recovery costs and transaction-identifier trials. A relevant incident report would strengthen practical motivation.
- **Limits** include one machine, forced failures, a fixed input and simple enrichment. Results will apply to those conditions, not all scientific pipelines.

4. Existing solutions

What solutions already exist?

Main point

The approaches make replay harmless or commit completion with data. Their guarantees depend on keys, versions and recovery logic.

Terms to define before comparing

- **Idempotent:** doing it twice gives the same result as doing it once. Merging task 4 twice, for example, still leaves exactly one copy of each earthquake.
- **UPSERT** means update or insert. MERGE implements a key-based match, updating existing rows and inserting new ones.

Comparison of approaches

Approach	How it works	Handles well	Limits under my conditions	Source
A. Plain append + separate marker (baseline)	Write rows with INSERT/append, then write a "done" file; on restart skip marked tasks	Simple baseline with little bookkeeping	Crash between the steps gives duplicates (save first) or missing rows (mark first)	Airflow
B. Idempotent upsert (MERGE on a record key)	Re-running a batch updates existing rows instead of adding copies	With stable keys and repeatable values, a replay avoids new copies	Needs stable keys; may read or rewrite data; concurrent conflicts may require retries; a premature marker can still skip unsaved rows	Airflow, delta-rs
C. Atomic progress in the commit (Delta transaction identifiers, appId + version)	The "done" record is written in the same log entry as the rows, so both commit or neither does	Commits data and task progress together; performance remains to be measured	Latest version per application identifier; ordering and retention assumptions need testing; client-specific	Delta protocol

			behavior matters	
D. Framework-managed exactly-once (Spark checkpoints + write-ahead log + idempotent sink)	Engine logs offsets per micro-batch and replays from the checkpoint after failure	Framework-managed recovery and progress	Needs Spark, a replayable source and Java; heavy for a one-student laptop study; guarantee depends on the sink	Spark guide

Conditions: one to three workers, one saved input and one local Delta table.

How the approaches differ

- Approach B makes save-then-mark safe to repeat, as long as each record has a stable key and enrichment gives the same values every time. It cannot help a pipeline that marks first, which can still lose output. Its extra reads may cost more; I have not measured that yet.
  https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html
- Approach C saves the data and the progress record in one commit. Before writing, the client checks whether the task's identifier is already recorded and skips the task if so. If parallel workers share one increasing version number and finish out of order, a task that never finished can be skipped.
  https://github.com/delta-io/delta/blob/master/PROTOCOL.md#transaction-identifiers
- Approach D's strength is that recovery is automatic: the engine records each batch's position in a checkpoint and replays from there after a failure, so developers do not write their own markers. Under my conditions, though, it needs input that can be replayed, an output that tolerates repeats, and Spark with Java, which is more than this local study needs.
  https://spark.apache.org/docs/latest/streaming/getting-started.html

Implications for the comparison

- Record-key idempotency and atomic task completion solve different parts of recovery. Neither alone proves that every input record was processed.
- Delta uses optimistic concurrency: each writer works without locking the table and checks for conflicts only when it commits. If another writer changed the same data first, the commit is rejected and must be retried. I will measure how often this happens rather than assume it.
  https://github.com/delta-io/delta-rs/blob/main/docs/how-delta-lake-works/delta-lake-acid-transactions.md

5. Key unknown

What is the most important thing you still do not know?

Main point: the central unknown

Which of MERGE and transaction identifiers preserves the reference dataset after the selected crashes, and what recovery cost does each incur? Correctness comes first; cost helps choose between implementations that pass.

Why this unknown matters most

- My synthetic dry run confirmed how the unsafe baselines fail and that merge survived one crash point, but transaction identifiers remain untested.
- No repeated timing evidence supports a performance ranking.

Experiment to resolve it

1. I will add task-scoped transaction identifiers and check progress before writing. Each task id will include the input file's version and the enrichment rules' version, so a marker from an old run cannot match new work. The reference answer is built in one simple process that enriches each input record once.
2. For each strategy, I will define the event order and terminate before writing, after files but before commit, and after commit but before marking. The mark-first baseline also needs termination after marking but before saving.
3. I will vary workers (1, 2, 3) and batch size (100, 250, 1,000), with no-crash controls.
4. Five repetitions per condition will use the same archived USGS file. A small targeted test will also replay one task concurrently.
5. Missing means absent reference keys; duplicates are extra copies per key; incorrect means wrong derived values on any copy. I will record retries, repeated enrichment calls and elapsed time, reporting medians and ranges.

Decision rule

If the result shows	**Then the design**
C passes all selected correctness checks and has lower measured recovery cost	Use C within its tested assumptions; retain B as a comparison
C fails a correctness check, or B performs better while both pass	Investigate the cause; choose B only if it passes the same checks
Both correct, similar cost	Prefer the simpler implementation and document the tradeoff

How findings influence design

A failure would point to a library or task-id problem to fix before choosing. Zero errors would show safety only under the tested conditions, not in general. Each conclusion will cite my written predictions and run records.
https://github.com/delta-io/delta/blob/master/PROTOCOL.md#transaction-identifiers

6. Individual scope

Your Individual Scope

What I will investigate

I will compare append-then-mark, mark-then-append, MERGE and transaction identifiers under worker crashes and restart, using one archived USGS snapshot and one local table.

What I will exclude

- Multi-machine clusters and Apache Spark
- Cloud object stores, including Amazon Simple Storage Service (S3)
- Streaming sources and network partitions
- Schema changes
- Realistic machine-learning enrichment

The comparison

Idempotent merge versus Delta transaction identifiers, with naive append and mark first as failing baselines. Judged on correctness (missing, duplicate, incorrect rows) and cost (commit retries, elapsed time).

Tools and data

Tool or data	Role
Python 3.10+, pandas, PyArrow	Language and data handling
deltalake package	Delta Lake tables without Java or Spark
Python multiprocessing	Parallel worker processes
pytest	Unit and end-to-end tests
Archived USGS input and SHA-256 fingerprint	Secure Hash Algorithm, 256-bit digest; verifies the saved file
Synthetic fixture	Repeatable tests without internet

Possible solution direction (not proven)

A possible direction is atomic task completion with versioned task identities, or a replay-safe MERGE followed by a marker. The choice remains provisional until both face equivalent correctness checks.

Why one student can do it

- It needs one laptop and public data, and the enrichment always gives the same output for the same input, so checking results is simple. I will archive the exact feed file, because its fingerprint alone cannot recreate it.
- Synthetic fixtures support repeatable failure scenarios without network access.
- A script will run the experiment grid. I will start with one worker and one crash, add the controls and replay tests next, and time a small pilot before the full grid. The goal is a bounded comparison, not a production system.

NSF 26-512 summary

Published opportunity: NSF 26-512

The NSF AI Datasets opportunity provides the published motivation for this study: preparing existing scientific data for AI-enabled use.
Primary reference: official solicitation, checked October 18, 2026.
https://www.nsf.gov/funding/opportunities/ai-datasets-unlocking-dataset-value-ai-enabled-scientific-discovery/nsf26-512/solicitation

Key facts

Item	Detail
Title	Unlocking Dataset Value for AI-Enabled Scientific Discovery (AI Datasets)
Number	NSF 26-512, published July 21, 2026
Full proposal deadline	November 4, 2026, 5 p.m. submitting organization's local time; annually thereafter on the first Wednesday of November
Total funding	$60 million to $100 million, subject to available funds
Expected awards	25 to 50
Contact	aidatasets@nsf.gov

Section by section

Section	What it says	What it means for A15
Summary of Program Requirements	Three award tiers; no letter of intent or preliminary proposal.	Funding relevance; no proof of a failure incident.
Introduction	Improve existing scientific datasets for AI discovery; new data collection is out of scope.	The earthquake feed is an existing dataset.
Program Description	Feature and metadata enhancement, reliable pipelines, and dataset augmentation or harmonization. Governance and integrity are expected.	Restart correctness addresses one integrity concern.

Award Information	Planning: $200,000 maximum. Impact: $2 million, up to three years. Flagship: $5 million, up to three years.	Award size does not establish project fit or success probability.
Eligibility	Eligible organizations submit proposals. University submissions act on behalf of faculty; institutional rules still apply.	Professor review precedes a decision on formal submission.
Proposal Preparation	Follow the current Proposal & Award Policies & Procedures Guide (PAPPG) and supplements. Planning guidance includes a concept outline, concurrence and an eight-page description. https://www.nsf.gov/policies/pappg/24-1/ch-2-proposal-preparation	The class paper follows the course rubric.
Proposal Review	Intellectual Merit and Broader Impacts, plus community benefit, enhancement value, integrity, adoption and sustainment.	These criteria guide a possible later grant proposal.
Award Administration	Standard award reporting; virtual and relevant in-person program meetings.	Administration is outside the paper's research question.
Agency Contacts	Program questions: aidatasets@nsf.gov.	Contact would be part of later proposal preparation.
Other Programs	Related infrastructure includes Integrated Data Systems and Services (IDSS) and the National Artificial Intelligence Research Resource (NAIRR).	Infrastructure choice remains outside the local benchmark.

Connection to the paper

- The opportunity supports the motivation for reliable preparation. It does not establish an observed restart failure.
- **The proposed comparison is a bounded study within that broader area.**
- A formal application would also need a scientific dataset benefit and community plan.

Sources and acronyms

Sources and acronyms

Sources

The paper uses the published opportunity, technical documentation for existing approaches, and additional implementation documentation. Full addresses also appear beside the relevant explanations.

Role	Source	Full web address	Supports	Status
Published opportunity	NSF 26-512, AI Datasets solicitation	https://www.nsf.gov/funding/opportunities/ai-datasets-unlocking-dataset-value-ai-enabled-scientific-discovery/nsf26-512/solicitation	Evidence	Official primary source checked October 18, 2026
Technical documentation, existing approach	Delta Lake protocol, "Transaction Identifiers"	https://github.com/delta-io/delta/blob/master/PROTOCOL.md#transaction-identifiers	Evidence; approach C	Primary protocol; recovery mechanism
Technical documentation, existing approach	Apache Airflow best practices, "Creating a task"	https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html	Evidence; approaches A and B	Primary documentation; paraphrased in the paper
Technical documentation, existing approach	Spark Structured Streaming guide, "Fault Tolerance Semantics"	https://spark.apache.org/docs/latest/streaming/getting-started.html	Evidence; approach D	Primary documentation; framework comparison
Additional technical material	delta-rs, "Delta Lake transactions"	https://github.com/delta-io/delta-rs/blob/main/docs/how-delta-lake-works/delta-lake-acid-transactions.md	Optimistic concurrency, why merges retry	Primary implementation documentation

Additional research (optional)	Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores", Proceedings of the Very Large Data Bases (VLDB) Endowment, 2020	https://vldb.org/pvldb/vol13/p3411-armbrust.pdf	Background for approach C	Optional background
Data	USGS earthquake CSV feeds	https://earthquake.usgs.gov/earthquakes/feed/v1.0/csv.php	Worked example	Primary data-format reference checked

Acronyms

Definitions are expanded at first use in the six numbered sections.

Acronym	**Meaning**
ACID	Atomicity, consistency, isolation, durability
AI	Artificial intelligence
CSV	Comma-separated values
DML	Data manipulation language
IDSS	Integrated Data Systems and Services (NSF program)
ML	Machine learning
MVCC	Multi-version concurrency control
NAIRR	National Artificial Intelligence Research Resource
NSF	National Science Foundation
PAPPG	Proposal & Award Policies & Procedures Guide (NSF)
PI	Principal investigator

S3	Simple Storage Service (Amazon)
SHA-256	Secure Hash Algorithm, 256-bit
UPSERT	Update or insert
USGS	United States Geological Survey